Documento de Diseño: Detección de diferencias en escena usando estimación de fondo monocular

Marco Alonso Rodríguez Villegas
Carné: 2020168039
Escuela de Ingeniería en Computadores
Instituto Tecnológico de Costa Rica, Cartago, Costa Rica
markorovi24@estudiantec.cr

I. Información del documento

Para no interrumpir la lectura, todas las tablas y figuras del documento se agrupan en el *Anexo de tablas y figuras*, al final. Cada sección las cita por su número y resume su contenido.

I-A. Portada, versión y estatus

Este documento es el Documento de Diseño (*Software Design Description*) del proyecto «Detección de diferencias en escena usando estimación de fondo monocular». Corresponde a la versión 1.0, fechada el 2 de octubre de 2026, con estado «En revisión / Aprobado preliminar». Sigue el estándar IEEE Std 1016-2009 [2] (cláusulas 4 y 5 y Anexo C). La Tabla I reúne estos metadatos.

I-B. Organización y autoría

El documento se elabora en el Instituto Tecnológico de Costa Rica (TEC), Escuela de Ingeniería en Computadores, Sede Central, Cartago, Costa Rica. Autor Marco Alonso Rodríguez Villegas (Carné: 2020168039). La persona supervisora es Luis Alberto Chavarría Zamora y el revisor académico es Luis Diego Noguera Mena.

I-C. Historia de cambios

La versión 1.0 es la versión inicial del diseño preliminar. Incluye cuatro perspectivas principales, las perspectivas complementarias y el apéndice de alternativas de diseño. La Tabla II registra la versión, la fecha, la descripción de la modificación y su autor. Las revisiones posteriores se agregarán como filas nuevas de esa tabla.

II. Introducción

II-A. Propósito

Este documento describe el diseño preliminar del software que detecta diferencias entre dos imágenes RGB monoculares de un mismo espacio interior, tomadas en momentos distintos («antes» y «después»). Su propósito es guiar la implementación del software de procesamiento de imágenes y de los modelos basados en *transformers*. Para ello define los módulos, sus responsabilidades, sus interacciones y el algoritmo que decide si un objeto desapareció o se desplazó. Está dirigido a la persona desarrolladora, a la persona supervisora y a quienes evalúen el diseño. El diseño satisface los requerimientos del sistema, identificados en el texto por su código (RF, RNF, RI, RR, RD, RA y RIS), redactados según ISO/IEC/IEEE 29148:2018 [1] con sintaxis EARS [3].

II-B. Alcance

El producto diseñado es una aplicación autocontenida en Python que ejecuta el siguiente *pipeline*:

1. **Captura:** carga de dos imágenes desde archivo o desde una cámara RGB monocular conectada (RI-001).
2. **Estimación de profundidad:** un mapa de profundidad por imagen, con un modelo preentrenado (Depth Anything [4], [5]).
3. **Comparación espacial:** alineación de las imágenes, normalización de escala, diferencia de profundidad y extracción de regiones de cambio.
4. **Clasificación:** cada cambio se clasifica como «desaparición» o «desplazamiento», y en este último caso se calcula la distancia en centímetros (RF-004, RF-005).
5. **Segmentación semántica:** identificación de la categoría del objeto con SegFormer [8] (RF-006, RF-007).
6. **Reporte:** presentación del resultado con resaltado sobre la imagen (RF-008, RNF-002).

Quedan fuera del alcance los sensores de profundidad dedicados (LiDAR, RGB-D), el seguimiento de video en tiempo real, las escenas exteriores, el entrenamiento de modelos desde cero y la persistencia en bases de datos.

II-C. Contexto

El producto se usará en la inspección de espacios interiores, como el control de inventario, la vigilancia de activos y la supervisión de áreas de trabajo. En estos casos la verificación manual es lenta y subjetiva, y los sensores de profundidad dedicados son costosos. El sistema ofrece una alternativa de bajo costo que usa únicamente una cámara monocular convencional (webcam o teléfono celular) y un computador con GPU. Se desarrolla en el TEC bajo la supervisión de Luis Alberto Chavarría Zamora.

II-D. Resumen del documento

La sección III define los términos usados y la sección IV identifica a los interesados. La sección V presenta cuatro perspectivas de diseño principales. Dos describen la estructura (Contexto y Composición) y dos el comportamiento (Interacción, y Dinámica de estados con Algoritmos). Esa sección incluye además las demás perspectivas de IEEE 1016, y cada una está documentada o marcada explícitamente como «No aplica» o «Se documentará posteriormente». El Apéndice A compara dos arquitecturas alternativas para el *pipeline* y justifica la seleccionada. El documento cierra con las referencias y el anexo de tablas y figuras.

III. Glosario

Un *Vision Transformer* (ViT) es una red basada en el mecanismo de atención que procesa la imagen como una secuencia de parches. Una imagen *RGB monocular* es una imagen en color tomada con una sola cámara, sin información de profundidad directa. Un *mapa de profundidad* asigna a cada píxel un valor asociado a su distancia a la cámara, y puede ser relativo (definido salvo escala y desplazamiento) o métrico (en metros). Un *bounding box* es el rectángulo que encierra un objeto o una región de cambio y se usa para resaltarlo en el reporte. La *VRAM* es la memoria de la GPU, que limita el tamaño de los modelos y de las imágenes procesables. La Tabla III lista estos y los demás términos del documento (segmentación semántica, homografía, RANSAC, retroproyección, parámetros intrínsecos, inferencia, AbsRel, mIoU y *pipeline*).

IV. Interesados

El diseño tiene cuatro interesados. El **autor**, Marco Alonso Rodríguez Villegas, diseña y desarrolla el producto y necesita un diseño implementable que cumpla los requerimientos. La **persona supervisora**, Luis Alberto Chavarría Zamora, evalúa la calidad técnica y la factibilidad de la solución y de sus alternativas. El **revisor académico**, Luis Diego Noguera Mena, verifica la aplicación de IEEE 1016 y los aspectos formales del documento. Los **usuarios finales** (personal de supervisión, inventario y mantenimiento) requieren resultados claros, en lenguaje no técnico y con pocas interacciones (RNF-001, RNF-003). La Tabla IV resume el rol y el interés de cada uno.

V. Perspectivas de diseño

Según IEEE 1016 [2], el diseño se describe mediante perspectivas. Se documentan cuatro perspectivas principales, dos de estructura y dos de comportamiento. Las de estructura son Contexto (subsección V-A, con diagramas de contexto y de casos de uso) y Composición (subsección V-B, con un diagrama de capas y paquetes). Las de comportamiento son Interacción (subsección V-C, con un diagrama de secuencia) y Dinámica de estados y Algoritmos (subsección V-D, con un diagrama de estados y un diagrama de flujo). Las ocho perspectivas restantes se tratan en la subsección V-E. La Tabla V resume las perspectivas y la notación empleada.

V-A. Perspectiva de contexto (estructura)

Esta perspectiva describe los servicios del producto y sus fronteras con el entorno.

V-A1. Diagrama de contexto del sistema: La Figura 1 muestra el sistema como una única caja con sus interfaces externas. La **Persona usuaria** entrega las imágenes «antes» y «después» (RI-001) y recibe el reporte con el resaltado del cambio y las advertencias (RI-003). La **Cámara RGB monocular** (webcam o celular) aporta capturas directas. El **sistema de archivos local** provee y recibe imágenes y resultados exportados. Los **modelos preentrenados** (profundidad y segmentación) se invocan mediante la API de PyTorch (RI-002). El sistema no se conecta a servicios en la nube, por lo que las imágenes se procesan localmente.

V-A2. Casos de uso: La Figura 2 presenta los casos de uso. El caso principal es **CU-01 Detectar diferencias entre dos escenas**, que incluye **CU-02 Cargar o capturar imágenes** (RF-001, RI-001). La cámara interviene como actor secundario únicamente en CU-02, cuando la persona elige capturar. **CU-03 Consultar historial** permite revisar las comparaciones de la sesión activa (RF-014).

V-A3. Especificación del caso de uso principal: La Tabla VI especifica CU-01 por completo. En síntesis, la persona usuaria selecciona la imagen «antes» y la imagen «después» y pulsa «Analizar», que son tres interacciones (RNF-003). El sistema valida las imágenes y su iluminación, selecciona GPU o CPU, estima la profundidad, alinea las imágenes y extrae las regiones de cambio. Luego segmenta semánticamente ambas imágenes, clasifica cada cambio, calcula la distancia en centímetros cuando corresponde y muestra el reporte, registrándolo en el historial de la sesión. La precondición es que el entorno Python con PyTorch y los pesos de los modelos estén disponibles localmente, y que las imágenes se tomen con posición y orientación de cámara similares (RIS-001). Como postcondición, el sistema muestra el reporte y libera la GPU o la CPU. Si falla, muestra un error y vuelve al estado inicial. Los flujos alternos cubren una imagen inválida o corrupta (RF-012), la baja iluminación (RF-013), la ausencia de GPU (RF-010), la ausencia de cambios (RF-011), la aparición de un objeto nuevo y un fallo de inferencia. La Tabla VI detalla cada paso y cada flujo.

V-B. Perspectiva de composición (estructura)

V-B1. Diagrama de capas y paquetes: La Figura 3 organiza el software en capas y paquetes. La **capa de presentación** contiene el paquete de Interfaz y Captura. El **núcleo de orquestación** coordina el flujo. Los tres paquetes de la **capa de procesamiento** (Inferencias PyTorch, Motor de Comparación Espacial y Módulo de Reporte) son independientes entre sí, y solo el núcleo de orquestación los invoca. Esto cumple RA-003: cada modelo puede sustituirse sin modificar los demás módulos. La **plataforma** agrupa las bibliotecas externas.

V-B2. Descripción de cada paquete:

- **Paquete 1, Interfaz y Captura.** Recibe las imágenes y presenta resultados. *ImageLoader* lee archivos y *CameraCapture* captura desde una cámara conectada (RI-

001). *ImageValidator* verifica que el archivo no esté corrupto, que la resolución sea al menos 640×480, que la escena sea un interior y que la iluminación sea suficiente (RF-012, RF-013, RR-002). *MainWindow* concentra la interacción con la persona (subsección V-E).

- **Paquete 2, Núcleo de orquestación.** *PipelineController* es la fachada que ejecuta el flujo en orden: validación, profundidad, comparación, segmentación, clasificación y reporte. Maneja los flujos alternos del caso de uso. *ProgressNotifier* informa el avance a la interfaz.
- **Paquete 3, Inferencias PyTorch.** *DepthEstimator* y *SemanticSegmenter* envuelven los modelos preentrenados. *DeviceManager* elige GPU o CPU (RF-009, RF-010) y *ModelRegistry* carga y libera los pesos de forma secuencial para reducir el uso de VRAM.
- **Paquete 4, Motor de Comparación Espacial.** No depende de PyTorch; opera sobre arreglos (NumPy y OpenCV). *ImageAligner* estima la homografía entre las imágenes. *DepthNormalizer* ajusta la escala y el desplazamiento de los mapas de profundidad. *ChangeDetector* extrae las regiones de cambio. *ChangeClassifier* decide entre desaparición y desplazamiento. *DisplacementCalculator* calcula la distancia en centímetros (RF-003 a RF-005).
- **Paquete 5, Módulo de Reporte.** *ChangeHighlighter* dibuja cajas o máscaras sobre la imagen (RNF-002). *ReportBuilder* compone el texto no técnico (RNF-001, RF-008) y *SessionHistory* guarda en memoria las comparaciones de la sesión (RF-014).

V-B3. Asignación de requerimientos: La Tabla VII indica qué requerimientos satisface cada paquete. El paquete de Interfaz y Captura atiende la carga, la validación y la usabilidad (RF-001, RF-012, RF-013, RNF-003). El núcleo atiende la ausencia de cambios (RF-011) y el tiempo total (RR-001). Las Inferencias atienden los modelos y el uso de GPU o CPU (RF-006, RF-007, RF-009, RF-010). El Motor de Comparación atiende la detección, la clasificación y la distancia (RF-003 a RF-005). El Módulo de Reporte atiende el reporte, el resaltado y el historial (RF-008, RNF-002, RF-014). La portabilidad (RA-002) y la mantenibilidad (RA-003) son transversales.

V-C. Perspectiva de interacción (comportamiento)

La Figura 4 muestra el diagrama de secuencia del caso CU-01 en el orden temporal en que ocurren los mensajes. Los pasos 1 a 3 corresponden a las tres interacciones de la persona usuaria (RNF-003). En el paso 4 la interfaz delega en el orquestador, que valida las imágenes y la iluminación (RF-012, RF-013). En los pasos 5 y 6 el orquestador pide a las Inferencias que seleccionen el dispositivo y estimen la profundidad, que corre en GPU o, si no hay GPU, en CPU (RF-009, RF-010). Con los mapas de profundidad (D_a, D_d) el motor de comparación hace la alineación espacial y detecta las regiones de cambio (paso 8). Si existen regiones, el orquestador solicita la segmentación semántica (paso 10) y luego la clasificación (paso 12). Por último, el módulo de reporte compone el resultado y la interfaz lo muestra (pasos 14 a 16). El orquestador es el único que invoca a los demás módulos, por lo que el motor de comparación permanece desacoplado de los modelos (RA-003).

V-D. Perspectivas de dinámica de estados y algoritmos (comportamiento)

V-D1. Dinámica de estados de la sesión: La Figura 5 describe los estados del sistema durante una comparación. En *Inactivo* el sistema espera. Al cargar imágenes pasa a *Esperando imágenes* y, con ambas cargadas y la orden de analizar, a *Validando*. Si la validación falla (imagen corrupta o que no es de un interior, RF-012) pasa a *Error*. Si tiene éxito, y con los avisos de CPU o baja luz cuando aplican (RF-010, RF-013), pasa a *Procesando*, donde se ejecutan en orden la estimación de profundidad, la comparación, la segmentación y la clasificación. Un fallo de inferencia lleva también a *Error*. Al terminar llega a *Reporte disponible*, y desde ahí vuelve a *Inactivo* cuando la persona inicia una nueva comparación o consulta el historial (RF-014). Desde *Error*, la persona acepta el mensaje y el sistema vuelve a *Inactivo*. La Tabla VIII detalla cada transición con su evento y su requerimiento.

V-D2. Algoritmo de decisión: desaparición o desplazamiento: La Figura 6 muestra el algoritmo de clasificación como diagrama de flujo. El algoritmo opera sobre las imágenes I_a (antes) e I_d (después) y sus mapas de profundidad D_a y D_d, con estos pasos:

1. **Alineación y normalización.** Se estima una homografía entre I_a e I_d (puntos característicos y RANSAC), y se normaliza la profundidad de D_d para que sea comparable con D_a:
$$D'_d(p) = \hat{s}\, D_d(p) + \hat{t}, \tag{1}$$
donde $\hat{s}$ y $\hat{t}$ se estiman de forma robusta sobre píxeles estáticos.
2. **Regiones de cambio.** Se calcula $\Delta D(p) = D'_d(p) - D_a(p)$ y se aplica un umbral τ_d sobre $|\Delta D|$. Tras operaciones morfológicas y componentes conexas se obtienen regiones de área al menos $A_{\text{mín}}$. El signo de la mediana de ΔD en cada región las separa en **vaciadas** V_i (positivo: el fondo quedó más lejos, había un objeto) y **ocupadas** O_j (negativo: ahora hay un objeto).
3. **Categoría.** SegFormer segmenta I_a e I_d. La categoría c_i de cada región es la etiqueta de interior más frecuente dentro de ella (RF-006, RF-007).
4. **Decisión.** Para cada V_i se busca una región O_j con la misma categoría y similitud de apariencia al menos τ_s. Si existe, es un **desplazamiento**. Si no, es una **desaparición**. Las O_j sin pareja se informan como apariciones no clasificadas.
5. **Distancia en centímetros.** Para un desplazamiento se retroproyectan los centroides de V_i y O_j, usando la mediana de profundidad Z dentro de cada máscara:
$$P = \Big(\tfrac{(u-c_x)Z}{f_x},\ \tfrac{(v-c_y)Z}{f_y},\ Z\Big), \tag{2}$$
$$d = 100\,\|P_d - P_a\|_2 \text{ [cm]}, \tag{3}$$

con Z en metros. Los parámetros intrínsecos (f_x, f_y, c_x, c_y) se obtienen de los metadatos de la imagen o de una calibración. Un desplazamiento solo se reporta si d supera τ_{mov}, para filtrar el ruido de la estimación.

V-D3. Parámetros y limitaciones: La Tabla IX lista los parámetros del algoritmo. Como valores iniciales se proponen: umbral de diferencia de profundidad τ_d de 5 % del rango de profundidad, área mínima $A_{\text{mín}}$ de 0.5 % del área de la imagen, similitud mínima de apariencia $\tau_s = 0{,}6$ y desplazamiento mínimo reportable $\tau_{mov} = 3$ cm. Estos valores se calibrarán en la fase de pruebas (RF-003 a RF-005).

La precisión en centímetros depende de la escala métrica de la profundidad estimada y de los parámetros intrínsecos. El diseño asume una cámara casi fija entre ambas capturas (RIS-001), por lo que la homografía solo corrige diferencias pequeñas de pose. La confiabilidad se verificará según RF-005 contra mediciones manuales.

V-E. Otras perspectivas de IEEE 1016

V-E1. Lógica: Se documentará en fases posteriores de implementación (diagrama de clases detallado). Las clases previstas son las nombradas en la Figura 3.

V-E2. Dependencias: La Tabla X resume las dependencias entre entidades. La interfaz depende del núcleo de orquestación y de OpenCV para la captura. El núcleo depende de los tres paquetes de procesamiento. Las Inferencias dependen de PyTorch [10], de Hugging Face Transformers [11] y, de forma opcional, de CUDA. El Motor de Comparación y el Módulo de Reporte dependen de OpenCV [12] y NumPy, sin depender de PyTorch. El grafo es acíclico: ningún paquete de la capa de procesamiento depende de otro de esa capa.

V-E3. Información (datos persistentes): **No aplica.** Los requerimientos del sistema no exigen una base de datos lógica. El historial de RF-014 reside en memoria durante la sesión y no se persiste.

V-E4. Uso de patrones: Se aplican cuatro patrones:

- **Estrategia**, en *DepthEstimator* y *SemanticSegmenter*, para sustituir modelos (RA-003, Apéndice A).
- **Fachada**, en *PipelineController*, como único punto de entrada al flujo.
- **Adaptador**, en las envolturas de los modelos de Hugging Face y PyTorch, para ofrecer una interfaz uniforme.
- **Observador**, en *ProgressNotifier*, para informar el avance a la interfaz.

V-E5. Interfaces: La Tabla XI lista las interfaces internas entre módulos, sin incluir las de usuario. Las Inferencias exponen `estimate(img)`, que devuelve un mapa de profundidad, y `segment(img)`, que devuelve un mapa de etiquetas. El Motor de Comparación expone `compare(Ia, Id, Da, Dd)`, que devuelve las regiones V_i y O_j, y `classify(regs, labels, K)`, que devuelve la lista de cambios con tipo, categoría y distancia. El Módulo de Reporte expone `build(cambios, Ia, Id)`, que devuelve el texto y la imagen resaltada. Las firmas son preliminares y se precisarán en la implementación.

V-E6. Interfaz de usuario: La interfaz de escritorio ofrece dos selectores («Antes» y «Después»), cada uno con las opciones «Cargar archivo» y «Capturar», un botón «Analizar», un panel de resultados y una lista de historial. Seleccionar las dos imágenes y pulsar «Analizar» son las tres interacciones del flujo (RNF-003). El panel usa lenguaje no técnico y resalta la región del cambio (RNF-001, RNF-002). La tecnología gráfica concreta *se documentará en fases posteriores de implementación*.

V-E7. Estructura: Se documentará en fases posteriores de implementación (estructura interna de cada paquete). La estructura de alto nivel está cubierta en la perspectiva de composición.

V-E8. Recursos: El recurso crítico es la VRAM de la GPU. Para limitarla, los modelos se cargan y liberan de forma secuencial (primero el de profundidad y luego el de segmentación), se usa precisión mixta en GPU y las inferencias se serializan. Si no hay GPU, se usa la CPU (RF-010). La memoria y el tiempo estimados se analizan en el Apéndice A. El resto del modelo de recursos *se documentará en fases posteriores de implementación*.

APÉNDICE A
ALTERNATIVAS DE DISEÑO (EVALUACIÓN DI4)

Este apéndice compara dos arquitecturas para el *pipeline* principal, que es el aspecto de diseño más crítico porque determina la precisión espacial, el tiempo de respuesta y la memoria requerida.

A-A. Alternativas consideradas

Alternativa A (seleccionada). Depth Anything para estimación de profundidad [4], [5], más SegFormer para segmentación semántica [8]. Depth Anything es un modelo basado en ViT entrenado con grandes volúmenes de imágenes, con variantes de distinto tamaño (≈25 M a 335 M de parámetros) y una variante de profundidad métrica para interiores. SegFormer usa un codificador jerárquico y un decodificador ligero, con variantes de ≈3.7 M (B0) a ≈82 M (B5) de parámetros.

Alternativa B. MiDaS [6], [7] para profundidad, más Mask2Former [9] para segmentación. MiDaS entrega profundidad relativa (inversa, definida salvo escala y desplazamiento), por lo que los centímetros requieren calibración externa (por ejemplo, un objeto de tamaño conocido en la escena). Mask2Former es un segmentador universal con máscaras por instancia, lo que facilitaría emparejar objetos entre capturas. A cambio, el modelo es más pesado.

A-B. Cuadro comparativo

La Tabla XII compara las alternativas según los cuatro criterios definidos. Los valores publicados provienen de las referencias. Los marcados con † son estimaciones de ingeniería que deben validarse con las pruebas de RR-001 y RR-002. En precisión de profundidad en interiores (NYUv2), A reporta un AbsRel cercano a 0.043 frente a 0.048 de B [4], y solo A ofrece una variante de profundidad métrica. En segmentación, B supera a A (mIoU en ADE20K de ≈56.1 frente a ≈51.0).

En latencia, A estima 1 a 2 s por par en GPU frente a 2 a 4 s de B, y ambas cumplen el límite de 15 s de RR-001. En memoria, A necesita ≈3 a 5 GB de VRAM frente a ≈6 a 9 GB de B. En robustez ante la iluminación, ambas son comparables y se complementan con la advertencia de baja luz (RF-013).

La Tabla XIII pondera los criterios según su efecto en los requerimientos. La precisión espacial pesa 35 % porque RF-005 exige la distancia en centímetros, la latencia 25 %, la memoria 20 % y la robustez 20 %. El total ponderado es 4.00 para A y 3.00 para B. Los pesos y los puntajes son un juicio de diseño que se revisará con los resultados de las pruebas.

A-C. Justificación técnica de la selección

Se selecciona la **Alternativa A** por las siguientes razones:

1. **Escala métrica.** La variante de interior de Depth Anything entrega profundidad en metros, que la ecuación (3) necesita. MiDaS solo da profundidad relativa y obligaría a introducir una calibración manual, lo que contradice la usabilidad del sistema (RNF-003).
2. **Cumplimiento de rendimiento y portabilidad.** A tiene menor latencia y menor consumo de VRAM, lo que facilita cumplir RR-001 en una GPU de gama media y mantiene la ejecución en CPU como contingencia (RF-010, RA-002).
3. **Escalabilidad.** Depth Anything y SegFormer ofrecen familias de modelos de distinto tamaño, lo que permite ajustar la precisión y el consumo al hardware disponible.
4. **Menor riesgo en la comparación.** La mayor precisión de Mask2Former no compensa su costo, porque el algoritmo de la Figura 6 solo necesita una etiqueta de categoría por región y no máscaras de instancia de alta calidad. Si las pruebas muestran errores de emparejamiento, la arquitectura permite sustituir el segmentador sin tocar los demás módulos (patrón Estrategia, RA-003).

Un riesgo conocido es que las licencias de los pesos (por ejemplo, algunos puntos de control de Depth Anything V2 y de SegFormer) pueden limitar el uso comercial. Para el alcance actual del proyecto no es una restricción, pero debe verificarse antes de cualquier uso posterior.

A-D. Impacto: aspectos de salud, seguridad, costo y ambiente

La Tabla XIV analiza los aspectos pedidos por el criterio DI4 y declara explícitamente los que no aplican. La alternativa seleccionada cumple los requerimientos RF-005, RF-010, RR-001, RA-002, RA-003, RD-003 y RD-004. No aplican la salud pública, la seguridad pública ni la cultura. Aplica el costo total de la vida, porque usa una cámara estándar en lugar de sensores LiDAR o RGB-D y requiere una GPU de gama media. Aplica en parte el carbono neto cero, porque se evita el entrenamiento de modelos y A consume menos cómputo que B, aunque no se ha medido. Aplican los recursos (menor uso de VRAM y RAM), la sociedad (las imágenes se procesan localmente por privacidad, y los mensajes son no técnicos) y, en parte, el ambiente (menor consumo de energía).

REFERENCIAS

[1] ISO/IEC/IEEE, *ISO/IEC/IEEE 29148:2018 – Systems and software engineering – Life cycle processes – Requirements engineering*. Geneva, Switzerland: ISO, 2018.

[2] IEEE, *IEEE Std 1016-2009, IEEE Standard for Information Technology – Systems Design – Software Design Descriptions*. New York, NY, USA: IEEE, 2009.

[3] A. Mavin, P. Wilkinson, A. Harwood, and M. Novak, "Easy approach to requirements syntax (EARS)," in *Proc. 17th IEEE Int. Requirements Eng. Conf. (RE'09)*, Atlanta, GA, USA, 2009, pp. 317–322.

[4] L. Yang, B. Kang, Z. Huang, X. Xu, J. Feng, and H. Zhao, "Depth Anything: Unleashing the power of large-scale unlabeled data," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2024.

[5] L. Yang, B. Kang, Z. Huang, Z. Zhao, X. Xu, J. Feng, and H. Zhao, "Depth Anything V2," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.

[6] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, "Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 3, pp. 1623–1637, 2022.

[7] R. Birkl, D. Wofk, and M. Müller, "MiDaS v3.1 – A model zoo for robust monocular relative depth estimation," arXiv:2307.14460, 2023.

[8] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, and P. Luo, "SegFormer: Simple and efficient design for semantic segmentation with transformers," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 12077–12090.

[9] B. Cheng, I. Misra, A. G. Schwing, A. Kirillov, and R. Girdhar, "Masked-attention mask transformer for universal image segmentation," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 1290–1299.

[10] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, pp. 8024–8035.

[11] T. Wolf *et al.*, "Transformers: State-of-the-art natural language processing," in *Proc. EMNLP: System Demonstrations*, 2020, pp. 38–45.

[12] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, 2000.

ANEXO DE TABLAS Y FIGURAS

Este anexo reúne todas las tablas y todas las figuras del documento, en el orden en que el texto las cita, para no interrumpir la lectura de las secciones. Primero se presentan las tablas (Tablas I a XIV) y a continuación las figuras (Figuras 1 a 6).

Tabla I
METADATOS DEL DOCUMENTO DE DISEÑO

Campo	**Valor**
Título	Documento de Diseño: Detección de diferencias en escena usando estimación de fondo monocular
Versión	1.0
Fecha de la versión	2 de octubre de 2026
Estado	En revisión / Aprobado preliminar
Organización	Instituto Tecnológico de Costa Rica (TEC), Escuela de Ingeniería en Computadores
Estándar aplicado	IEEE Std 1016-2009 (cláusulas 4 y 5, Anexo C) [2]

Tabla II
HISTORIA DE CAMBIOS

Versión	**Fecha**	**Descripción de modificaciones**	**Autor**
1.0	02/10/2026	Versión inicial del diseño preliminar, con cuatro perspectivas principales y apéndice de alternativas de diseño.	M. A. Rodríguez V.

Tabla III
GLOSARIO DE TÉRMINOS

Término	**Definición**
ViT (*Vision Transformer*)	Arquitectura basada en el mecanismo de atención, adaptada al procesamiento de imágenes. Divide la imagen en *parches* que se procesan como una secuencia.
RGB monocular	Imagen en color (rojo, verde, azul) capturada con una única cámara, sin información estereoscópica ni de profundidad directa.
Mapa de profundidad	Matriz en la que cada píxel almacena un valor asociado a la distancia entre la cámara y el punto de la escena. Puede ser relativo (a menos de escala y desplazamiento) o métrico (en metros).
Estimación de profundidad monocular	Inferencia, a partir de una sola imagen RGB, de un mapa de profundidad.
Segmentación semántica	Asignación de una categoría a cada píxel de una imagen.
Bounding box (caja delimitadora)	Rectángulo alineado a los ejes que encierra la región de un objeto o de un cambio, usado para resaltarlo en el reporte.
Homografía	Transformación proyectiva 3×3 que alinea dos vistas de una superficie plana o de una cámara con rotación pequeña.
RANSAC	Método robusto de estimación de parámetros que descarta correspondencias atípicas.
Retroproyección	Cálculo de un punto 3D a partir de un píxel, su profundidad y los parámetros intrínsecos de la cámara.
Parámetros intrínsecos	Distancias focales (f_x, f_y) y centro óptico (c_x, c_y) de la cámara.
Inferencia	Ejecución de un modelo ya entrenado sobre datos nuevos, sin modificar sus parámetros.
VRAM	Memoria de video de la GPU, que limita el tamaño de los modelos y de las imágenes procesables.
AbsRel	Error relativo absoluto medio de la profundidad estimada (menor es mejor).
mIoU	Intersección sobre unión media entre segmentaciones predichas y de referencia (mayor es mejor).
Pipeline	Secuencia ordenada de etapas (captura, inferencia, comparación, reporte).

Tabla IV
INTERESADOS (*stakeholders*) DEL DISEÑO

Interesado	**Rol**	**Interés en el diseño**
Marco Alonso Rodríguez Villegas	Autor, diseñador y desarrollador	Contar con un diseño implementable que cumpla los requerimientos.
Luis Alberto Chavarría Zamora	Persona supervisora	Evaluar la calidad técnica y la factibilidad de la solución y de las alternativas.
Luis Diego Noguera Mena	Revisor académico	Verificar la aplicación de IEEE 1016 y los aspectos formales.
Usuarios finales	Personal de supervisión, inventario y mantenimiento	Resultados claros, en lenguaje no técnico, con pocas interacciones (RNF-001, RNF-003).

Tabla V
PERSPECTIVAS SELECCIONADAS Y NOTACIÓN

Perspectiva	**Tipo**	**Notación**	**Sec.**
Contexto	Estructura	Diagrama de contexto y casos de uso UML	V-A
Composición	Estructura	Diagrama de capas y paquetes UML	V-B
Interacción	Comportamiento	Diagrama de secuencia UML	V-C
Dinámica de estados y algoritmos	Comportamiento	Diagrama de estados UML y diagrama de flujo	V-D
Otras (ocho restantes)	Varias	Texto y tablas	V-E

Tabla VI
ESPECIFICACIÓN DEL CASO DE USO PRINCIPAL CU-01

Campo	**Descripción**
Nombre	CU-01 Detectar diferencias entre dos escenas interiores.
Actores	Principal: Persona usuaria. Secundario: Cámara RGB monocular (solo si se captura directamente).
Descripción general	La persona usuaria entrega una imagen «antes» y una «después» de un mismo espacio interior. El sistema determina si hubo un cambio, lo clasifica como desaparición o desplazamiento, calcula la distancia en centímetros cuando corresponde, identifica la categoría del objeto y presenta un reporte.
Requerimientos asociados	RF-001 a RF-013, RNF-001 a RNF-003, RI-001 a RI-003, RR-001, RR-002.
Entradas	Dos imágenes RGB monoculares de resolución mínima 640×480 (RR-002), desde archivo o cámara.
Salidas	Reporte con: existencia de cambio, tipo de cambio, categoría del objeto, distancia de desplazamiento (cm), imágenes originales, región resaltada y advertencias.
Precondiciones	(1) El entorno Python con PyTorch y los pesos de los modelos están disponibles localmente. (2) La persona dispone de dos imágenes del mismo espacio, tomadas con posición y orientación de cámara similares (RIS-001). (3) El sistema está en estado *Inactivo*.
Postcondiciones	Éxito: se muestra el reporte, la comparación se agrega al historial de la sesión (RF-014) y los recursos de GPU/CPU quedan liberados. Fallo: se muestra un mensaje de error descriptivo, no se agrega nada al historial y el sistema vuelve a *Inactivo*.
Flujo principal	1) La persona selecciona la imagen «antes» (CU-02). 2) La persona selecciona la imagen «después» (CU-02). 3) La persona pulsa «Analizar» (tercera interacción, RNF-003). 4) El sistema valida ambas imágenes y su iluminación. 5) El sistema selecciona el dispositivo de cómputo (GPU si está disponible). 6) El sistema estima el mapa de profundidad de cada imagen. 7) El sistema alinea las imágenes, normaliza la escala y extrae las regiones de cambio. 8) El sistema segmenta semánticamente ambas imágenes y asigna una categoría a cada región. 9) El sistema clasifica cada cambio y calcula la distancia en centímetros si es un desplazamiento. 10) El sistema muestra el reporte con el resaltado y registra la comparación en el historial.
Flujos alternos	A1 (RF-012): una imagen es corrupta o no es de un interior válido. El sistema rechaza el procesamiento y muestra un error. A2 (RF-013): hay baja iluminación. El sistema continúa pero advierte que el resultado puede no ser confiable. A3 (RF-010): no hay GPU. El sistema ejecuta en CPU y avisa que el proceso será más lento. A4 (RF-011): no hay diferencias significativas. El sistema informa «sin cambios» y omite los pasos 8 y 9. A5: hay una región «ocupada» sin pareja (objeto nuevo). Se informa como aparición no clasificada, fuera del alcance de RF-004. A6: falla la inferencia (por ejemplo, memoria insuficiente). El sistema muestra un error y vuelve a *Inactivo*.

Tabla VII
ASIGNACIÓN DE REQUERIMIENTOS A PAQUETES

Paquete	**Requerimientos principales**
1. Interfaz y Captura	RF-001, RF-012, RF-013, RNF-003, RI-001, RR-002, RIS-001 a RIS-003
2. Núcleo de orquestación	RF-002, RF-011, RR-001, RA-001
3. Inferencias PyTorch	RF-002, RF-006, RF-007, RF-009, RF-010, RI-002, RD-001 a RD-004
4. Motor de Comparación Espacial	RF-003, RF-004, RF-005, RD-005
5. Módulo de Reporte	RF-008, RF-014, RNF-001, RNF-002, RI-003
Transversal	RA-002 (portabilidad), RA-003 (mantenibilidad)

Tabla VIII
TRANSICIONES DEL DIAGRAMA DE ESTADOS

Origen	**Destino**	**Evento o condición**	**Req.**
Inactivo	Esperando imágenes	La persona carga o captura una imagen	RF-001
Esperando imágenes	Validando	Hay dos imágenes y se pulsa «Analizar»	RNF-003
Validando	Procesando	Imágenes válidas; se emiten avisos de CPU o baja luz si aplica	RF-010, RF-013
Validando	Error	Imagen corrupta o no interior	RF-012
Procesando	Reporte disponible	Se completa la clasificación y el reporte	RF-008
Procesando	Error	Fallo de inferencia o memoria insuficiente	—
Reporte disponible	Inactivo	Nueva comparación o consulta de historial	RF-014
Error	Inactivo	La persona acepta el mensaje de error	RF-012

Tabla IX
PARÁMETROS DEL ALGORITMO (VALORES INICIALES POR CALIBRAR)

Símbolo	**Valor inicial**	**Significado**
τ_d	5 % del rango de profundidad	Umbral de diferencia de profundidad
$A_{\text{mín}}$	0.5 % del área de la imagen	Área mínima de una región de cambio
τ_s	0.6	Similitud mínima de apariencia entre V_i y O_j
τ_{mov}	3 cm	Desplazamiento mínimo reportable

Tabla X
DEPENDENCIAS ENTRE ENTIDADES

Entidad	**Depende de**
Interfaz y Captura	Núcleo de orquestación; OpenCV (captura)
Núcleo de orquestación	Inferencias, Motor de Comparación y Reporte
Inferencias PyTorch	PyTorch [10], Hugging Face Transformers [11], CUDA (opcional)
Motor de Comparación	OpenCV [12], NumPy (sin PyTorch)
Módulo de Reporte	OpenCV (dibujo de cajas y máscaras), NumPy

Tabla XI
INTERFACES INTERNAS (FIRMAS PRELIMINARES)

Interfaz	**Descripción**
`estimate(img)`	Entrada: imagen RGB. Salida: mapa de profundidad $H \times W$.
`segment(img)`	Entrada: imagen RGB. Salida: mapa de etiquetas $H \times W$.
`compare(Ia, Id, Da, Dd)`	Salida: lista de regiones V_i y O_j con máscara y signo.
`classify(regs, labels, K)`	Entrada: regiones, etiquetas e intrínsecos K. Salida: lista de cambios (tipo, categoría, d cm).
`build(cambios, Ia, Id)`	Salida: reporte (texto) e imagen con la región resaltada.

Tabla XII
COMPARACIÓN CUANTITATIVA Y CUALITATIVA DE LAS ALTERNATIVAS A Y B

Criterio	**Alternativa A: Depth Anything + SegFormer**	**Alternativa B: MiDaS + Mask2Former**	**Observación**
1. Precisión métrica y espacial (profundidad)	AbsRel en NYUv2 (interior, *zero-shot*) ≈0.043 (ViT-L); variante métrica de interior disponible	AbsRel en NYUv2 ≈0.048 (MiDaS v3.1 ViT-L); solo profundidad relativa	Valores reportados en [4]; B requiere calibración de escala para obtener centímetros
1b. Precisión de segmentación (mIoU en ADE20K)	SegFormer-B5 ≈51.0	Mask2Former Swin-L ≈56.1	Mayor precisión de B en segmentación; pesa menos para este criterio
2. Latencia por par completo, GPU†	≈1–2 s	≈2–4 s	Ambas bajo el límite de 15 s (RR-001)
2b. Latencia por par completo, CPU†	≈20–45 s (variantes pequeñas: menos)	≈60–120 s	CPU solo como contingencia (RF-010)
3. Parámetros totales (aprox.)	≈420 M (variantes pequeñas desde ≈30 M)	≈560 M	A permite escalar el tamaño según el hardware
3b. VRAM pico, carga secuencial†	≈3–5 GB	≈6–9 GB	B exige GPU de mayor gama
4. Robustez ante variaciones de iluminación (interior)	Alta: preentrenamiento a gran escala con datos diversos; SegFormer reporta buena robustez a corrupciones [8]	Media a alta: Mask2Former con Swin es robusto, MiDaS usa menos diversidad de datos	Ambas se complementan con la advertencia de baja luz (RF-013)

Tabla XIII
MATRIZ PONDERADA DE DECISIÓN (PUNTAJE DE 1 A 5)

Criterio	**Peso**	**A**	**B**
Precisión métrica y espacial	35 %	4	3
Latencia	25 %	4	3
Memoria (VRAM y RAM)	20 %	4	2
Robustez ante iluminación	20 %	4	4
Total ponderado	100 %	**4.00**	**3.00**

Tabla XIV
ANÁLISIS DE ASPECTOS DE LA ALTERNATIVA SELECCIONADA (DI4)

Aspecto	**¿Aplica?**	**Análisis**
Cumplimiento de requerimientos	Sí	A cumple RF-005, RF-010, RR-001, RA-002, RA-003, RD-003 y RD-004 (Tabla XII).
Salud pública	No aplica	El sistema no procesa datos biomédicos ni afecta la salud de las personas.
Seguridad pública	No aplica	No es un sistema de seguridad crítica; solo registra cambios en objetos de interiores.
Costo total de la vida (ciclo de vida)	Sí	Usa una cámara estándar en lugar de sensores LiDAR o RGB-D y requiere una GPU de gama media por su menor VRAM, lo que reduce el costo de adquisición.
Carbono neto cero	Sí, parcial	Reutilizar modelos preentrenados evita el entrenamiento (RD-003) y A consume menos cómputo por inferencia que B. No se ha medido el consumo energético.
Recursos	Sí	A reduce el uso de VRAM y RAM, y puede usar variantes pequeñas en equipos limitados. Los modelos son públicos.
Cultura	No aplica	El producto no maneja contenido cultural; la interfaz está en español.
Sociedad	Sí	Las imágenes de interiores pueden mostrar personas o información sensible, por lo que se procesan localmente y no se envían a la nube. Los mensajes usan lenguaje no técnico (RNF-001).
Ambiente	Sí, parcial	Menos cómputo y no entrenar modelos reducen el consumo de energía, aunque sin cuantificar.

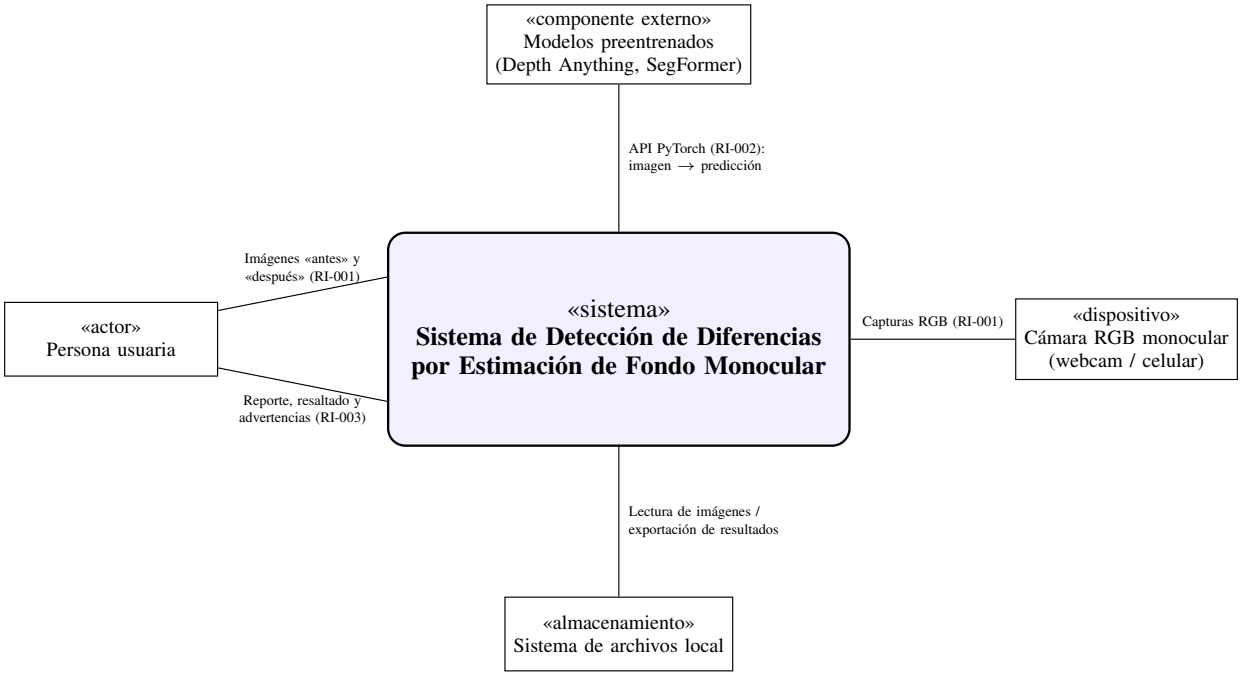


Figura 1. Diagrama de contexto del sistema, con las fronteras y las interfaces externas con la persona usuaria y la cámara RGB monocular.

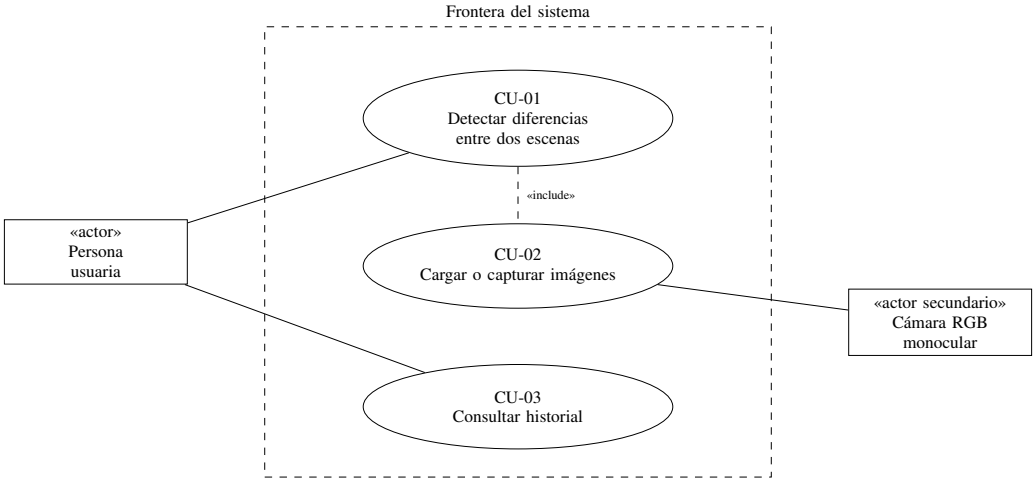


Figura 2. Diagrama de casos de uso. CU-01 incluye CU-02 y la cámara es un actor secundario de este último.

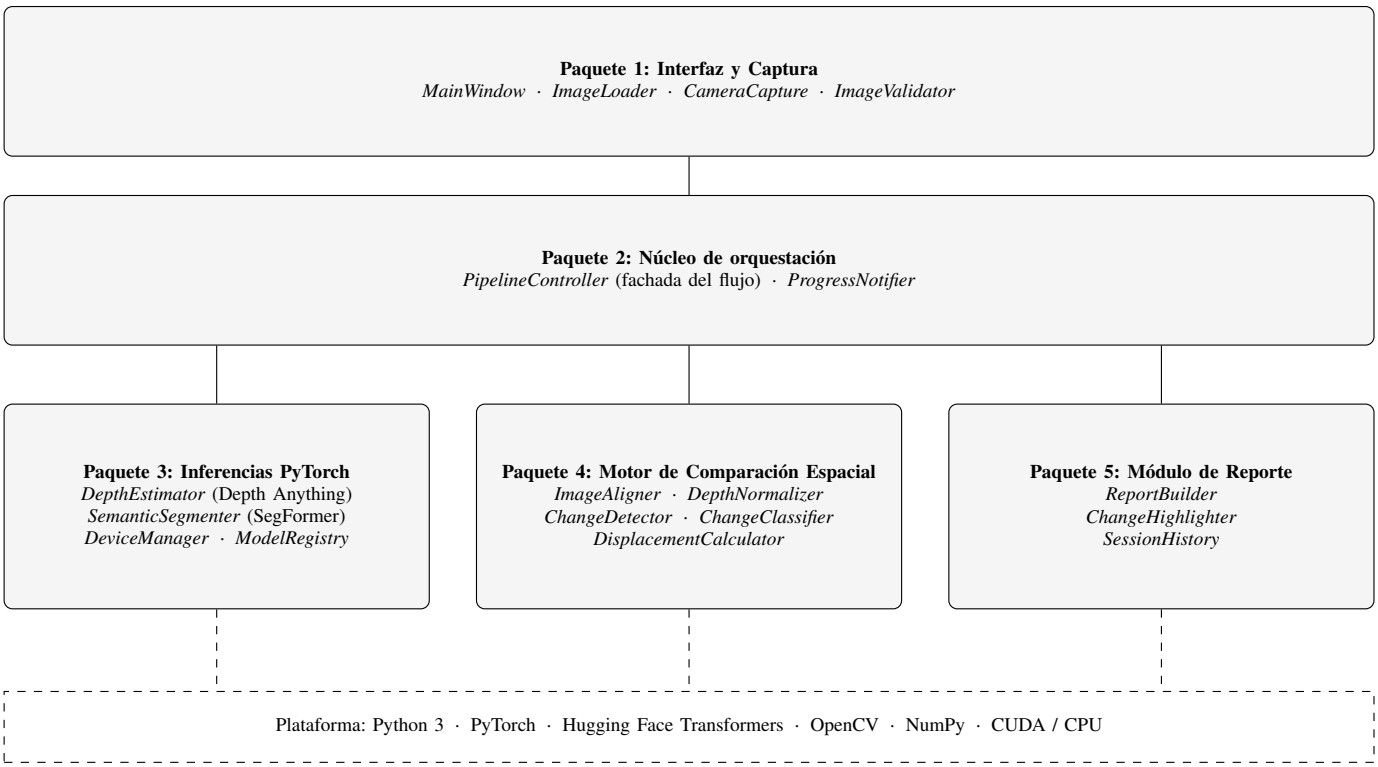


Figura 3. Diagrama de capas y paquetes del software. Las flechas continuas indican invocación entre paquetes y las discontinuas, dependencia de la plataforma.

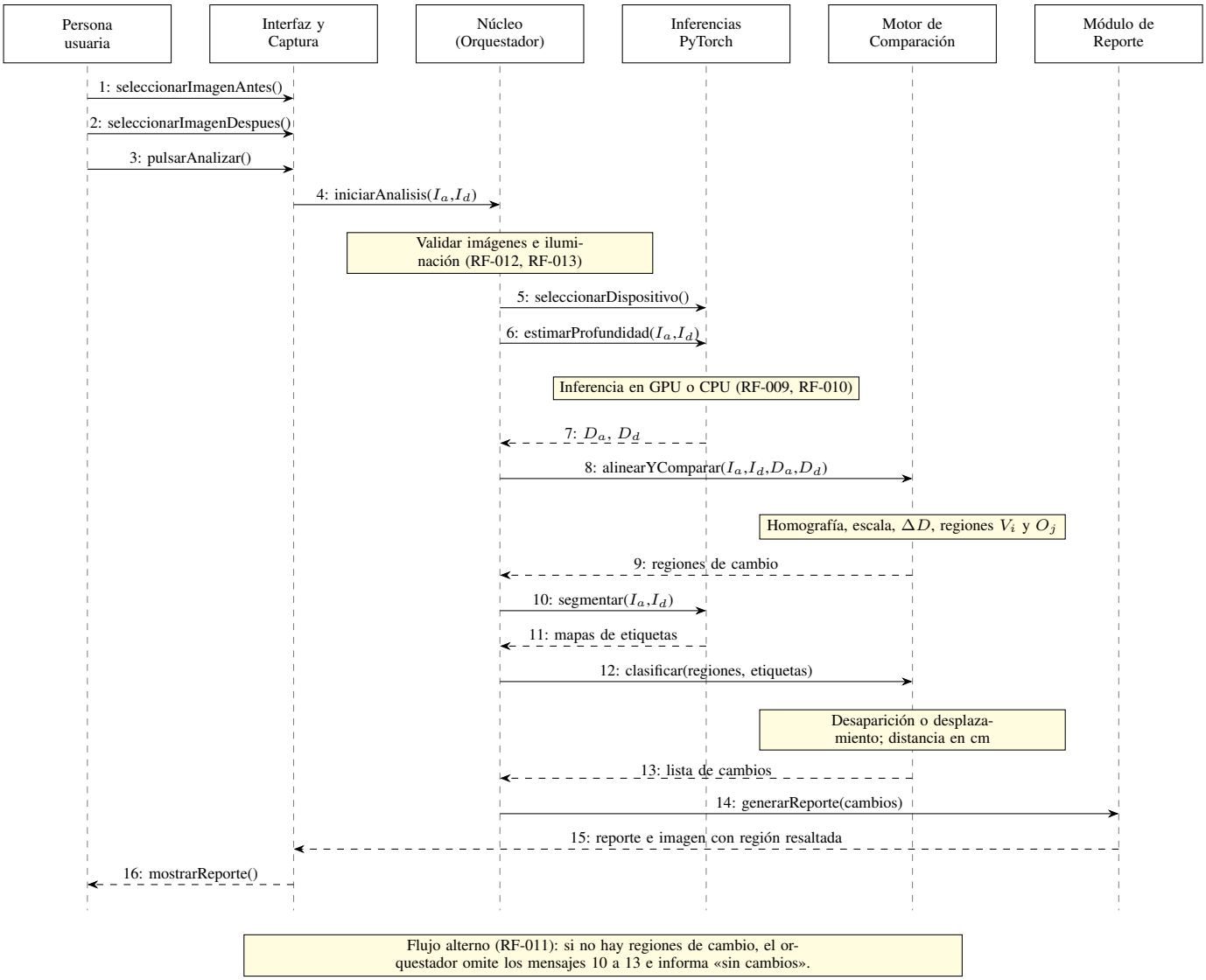


Figura 4. Diagrama de secuencia UML del caso de uso CU-01. Las flechas continuas son invocaciones y las discontinuas, retornos.

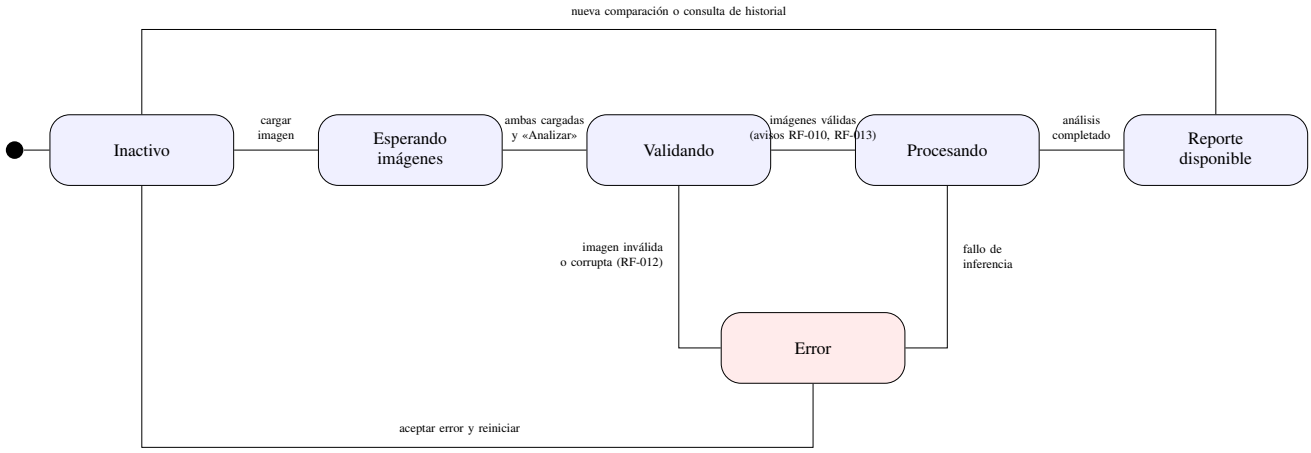


Figura 5. Diagrama de transición de estados UML de la sesión de comparación.

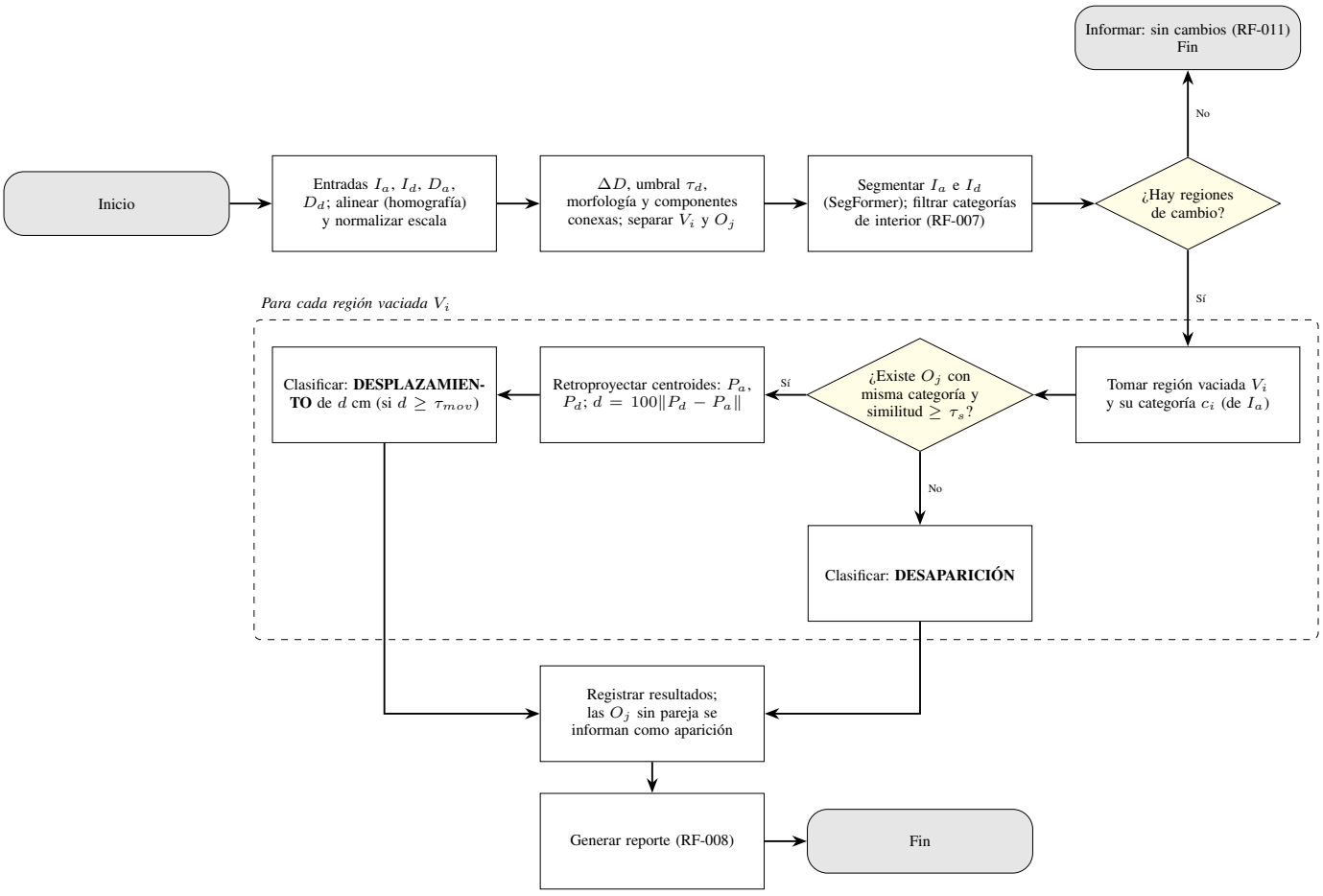


Figura 6. Diagrama de flujo del algoritmo que clasifica un cambio como desaparición o desplazamiento y calcula la distancia en centímetros.